

POS Lernaufgabe - REST API: Projektverwaltung

Trainingsangabe im Stil der Blogverwaltungs-Aufgabe. Die Aufgaben sind absichtlich nicht bis ins letzte Detail ausformuliert: Entscheide selbst, welche DTOs, Mapper und Service-Methoden sinnvoll sind.

Technische Vorgaben

- Verwende entweder Minimal API oder Controller.
- Verwende kein Repository-Pattern.
- LINQ-Abfragen befinden sich im Service.
- Das Service darf direkt mit dem DbContext arbeiten.
- DTOs und Mapper dürfen verwendet werden, sind aber nicht verpflichtend.
- Die API soll sinnvolle HTTP-Statuscodes zurückgeben.
- Eingaben sollen einfach validiert werden.
- Mocking ist bei Tests erlaubt.
- Die bestehende Struktur des Projekts soll beibehalten werden.

Vorgegebene Datenbasis

Erstelle eine neue SQLite-Datenbank, z. B. Projects.db. Die Klassen dürfen erweitert werden, die Grundstruktur soll aber erhalten bleiben. Nutze sinnvolle Seed-Daten, damit alle Filter und Statistiken überprüfbar sind.

```
namespace Aufgabe3_RestfulApi.Model;

public class Project
{
    public Project(string name, string customer, string status, DateTime deadline)
    {
        Name = name;
        Customer = customer;
        Status = status;
        Deadline = deadline;
    }

    public int Id { get; private set; }
    public string Name { get; set; }
    public string Customer { get; set; }
    public string Status { get; set; } // z. B. Planned, Active, Done
    public DateTime Deadline { get; set; }
    public List<TaskItem> Tasks { get; } = new();
}

public class TaskItem
{
    public TaskItem(string title, string description, string priority,
        string state, DateTime dueDate)
    {
        Title = title;
        Description = description;
        Priority = priority; // z. B. Low, Medium, High
        State = state; // z. B. Open, InProgress, Done
        DueDate = dueDate;
    }

    public int Id { get; private set; }
    public string Title { get; set; }
    public string Description { get; set; }
    public string Priority { get; set; }
    public string State { get; set; }
    public DateTime DueDate { get; set; }

    public Project Project { get; set; } = null!;
    public Employee? Assignee { get; set; }
}
```

```

        public List<Skill> RequiredSkills { get; } = new();
    }

    public class Employee
    {
        public Employee(string fullName, string department)
        {
            FullName = fullName;
            Department = department;
        }

        public int Id { get; private set; }
        public string FullName { get; set; }
        public string Department { get; set; }
        public OfficeAddress Office { get; set; } = null!;
        public List<TaskItem> Tasks { get; } = new();
    }

    public record OfficeAddress(string Street, string City, string Country);

    public class Skill
    {
        public Skill(string code, string name)
        {
            Code = code;
            Name = name;
        }

        public int Id { get; private set; }
        public string Code { get; set; }
        public string Name { get; set; }
        public List<TaskItem> Tasks { get; } = new();
    }

    using Aufgabe3_RestfulApi.Model;
    using Microsoft.EntityFrameworkCore;

    namespace Aufgabe3_RestfulApi.Infrastructure;

    public class ProjectsContext : DbContext
    {
        public ProjectsContext(DbContextOptions<ProjectsContext> options) : base(options)
        {
        }

        public DbSet<Project> Projects => Set<Project>();
        public DbSet<TaskItem> TaskItems => Set<TaskItem>();
        public DbSet<Employee> Employees => Set<Employee>();
        public DbSet<Skill> Skills => Set<Skill>();

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<Employee>().OwnsOne(e => e.Office);

            modelBuilder.Entity<Project>()
                .HasMany(p => p.Tasks)
                .WithOne(t => t.Project)
                .IsRequired();

            modelBuilder.Entity<TaskItem>()
                .HasMany(t => t.RequiredSkills)
                .WithMany(s => s.Tasks);
        }
    }

```

Aufgabe 1: Projektverwaltung

Implementiere bzw. erweitere die Projektverwaltung.

Folgende Endpunkte sollen vorhanden sein:

- GET /api/projects
- GET /api/projects/{id}
- POST /api/projects
- PUT /api/projects/{id}
- DELETE /api/projects/{id}

Zusätzlich soll GET /api/projects optional nach Projektname oder Kunde filtern können.

GET /api/projects?search=shop

Die Rückgabe soll nicht die vollständigen Entity-Objekte enthalten, sondern eine geeignete reduzierte Darstellung.

Aufgabe 2: Paginierte Projektliste

Erweitere das Service um eine Methode, mit der Projekte seitenweise abgefragt werden können.

Endpoint:

GET /api/projects/paged?page=1&pageSize=10&search=shop&status=Active

Die Antwort soll mindestens enthalten:

- aktuelle Seite
- Seitengröße
- Gesamtanzahl der passenden Einträge
- Liste der Projekte

Ungültige Werte für page und pageSize sollen sinnvoll behandelt werden. Der Parameter status ist optional.

Aufgabe 3: Projektstatistik

Erstelle eine Statistik über Projekte.

Endpoint:

GET /api/projects/stats?minTasks=3&skill=CSharp

Die Statistik soll pro Projekt sinnvolle Informationen liefern, zum Beispiel:

- Projektname
- Kunde
- Anzahl aller Tasks
- Anzahl offener und erledigter Tasks
- Anzahl verschiedener Mitarbeiterinnen und Mitarbeiter
- früheste offene Deadline
- benötigte Skills

Die Parameter minTasks und skill sind optional. Die Filterung muss im Service mit LINQ erfolgen.

Aufgabe 4: Offene Tasks

Erstelle einen Endpunkt, der offene bzw. aktuelle Tasks liefert.

Endpoint:

```
GET /api/tasks/open?dueBefore=2026-12-31&priority=High&take=10
```

Die Rückgabe soll Tasks nach Fälligkeitsdatum aufsteigend sortieren. Optional kann nach Deadline, Priorität und maximaler Anzahl gefiltert werden.

Die Antwort soll eine kurze Darstellung enthalten, zum Beispiel Tasktitel, Projektname, Priorität, Status, Fälligkeitsdatum und zugewiesene Person.

Aufgabe 5: Mitarbeiter nach Abteilung

Erstelle einen Endpunkt, der Mitarbeiterinnen und Mitarbeiter einer bestimmten Abteilung liefert.

Endpoint:

```
GET /api/employees/by-department?department=Development&minOpenTasks=1
```

Die Antwort soll pro Person eine kurze Übersicht enthalten, zum Beispiel:

- Name
- Abteilung
- Land des Büros
- Anzahl offener Tasks
- Projekte, in denen die Person aktuell Aufgaben hat

Der Parameter department ist verpflichtend. minOpenTasks ist optional.

Aufgabe 6: Skill-Suche

Erstelle einen Endpunkt, der Skills mit ihren Einsätzen in Tasks auswertet.

Endpoint:

```
GET /api/skills/usage?search=net&minTasks=2
```

Die Rückgabe soll pro Skill mindestens den Skillnamen, den Code, die Anzahl der Tasks und eine kurze Liste betroffener Projekte enthalten. Beide Parameter sind optional.

Hinweise

- Lege Seed-Daten an, die mehrere Statuswerte, Prioritäten, Abteilungen, Länder und Skills enthalten.
- Achte darauf, dass Include/ThenInclude nur dort verwendet wird, wo Navigationsdaten wirklich benötigt werden.
- Validiere Pflichtfelder bei POST/PUT einfach, aber nachvollziehbar.
- Gib bei nicht gefundenen Datensätzen oder ungültigen Parametern passende HTTP-Statuscodes zurück.
- Tests können wahlweise mit gemocktem Service oder mit Testdatenbank geschrieben werden.